Taxpal

Description

Taxpal is a personal finance and tax estimator designed especially for freelancers and gig workers. Unlike salaried employees, freelancers often do not have access to structured payroll systems that handle taxes automatically. They need to manually track income from multiple sources, manage their expenses, and calculate how much they owe each quarter. This process can be time-consuming, confusing, and prone to errors. Taxpal addresses this challenge by providing a simple, web-based platform where users can log their income and expenses, categorize transactions, set monthly budget limits, and automatically receive regional tax estimates. It also generates detailed financial summaries that can be downloaded as reports, making it easier for users to file their taxes at the end of each financial period.

Purpose of the project

The main purpose of Taxpal is to simplify financial management for freelancers by combining three essential needs into one tool: **tracking, budgeting, and tax estimation**.

- First, it allows users to record their income and expenses, giving them a clear picture of their cash flow.
- Second, it enables budgeting by letting users categorize their transactions and set spending limits for different areas such as rent, utilities, or food. This helps them stay disciplined and avoid overspending
- Third, Taxpal includes a tax estimation engine that automatically calculates how much tax a user owes based on their income bracket and country of residence. The application even reminds users about quarterly due dates, so they never miss a payment.

By combining these functions in one place, Taxpal reduces the hassle of switching between spreadsheets, budgeting apps, and tax calculators. Instead, users get a complete solution that is intuitive, efficient, and reliable.

Getting Started

Before you can run Taxpal on your system, a few tools and services need to be set up. Think of this section as your quick start guide. By following these steps in order, you'll be able to get the application running smoothly on your machine.

Prerequisites

Before running the project, make sure you have the following:

1. **Node.js and npm** – These are required to install and run the project dependencies. You can check if Node.js is installed by typing node -v in your terminal. If it shows a version number, you're good to go. If not, download it from the Node.js official website.

2. **Git** – You'll need Git to clone the project repository. Check if it's installed by running git –v.

Installation

To install the project, follow these steps:

- **Clone the Repository**

  git clone https://github.com/springboardmentor0316/Taxpal_team1.git

  cd Taxpal/frontendd

  cd Taxpal/backend

- **Install Dependencies**
  Inside the project folder, run:

  npm install

  This will download and set up all the necessary packages.

Running the application

After installation is complete, you can start the project with: **npm run dev**

This command will launch both the frontend and backend (depending on your configuration).

- The frontend is usually available at **http://localhost:3000**
- Backend API: Accessible at http://localhost:1000 for testing endpoints.

Environment Setup for TaxPal Project

To run the project seamlessly, you need to set up environment variables for both the backend and

frontend. Create a .env file in the root directory of the backend project and add the required

configurations as shown below.

Required Environment Variables

Variable Name	Description	Example Value
USER_DB_URL	Connection string for the user database.	mongodb://localhost:27017/users
BUDGET_DB_URL	Connection string for the budget database.	mongodb://localhost:27017/budget
TRANSACTION_DB_URL	Connection string for the transaction database.	mongodb://localhost:27017/ttransaction
TAXESTIMATOR_DB_URL	Connection string for the taxestimator database.	mongodb://localhost:27017/taxestimator

TAXCALENDER_DB_URL	Connection string for the taxcalender database.	mongodb://localhost:27017/taxcalender
REPORT_DB_URL	Connection string for the report database.	mongodb://localhost:27017/report
JWT_SECRET	Secret key for signing JWT tokens.	supersecretkey123
SMTP_HOST	SMTP host for sending emails.	smtp.gmail.com
SMTP_PORT	SMTP port for the email server.	587
SMTP_USER	Email address used for sending emails.	your-email@example.com
SMTP_PASS	Password or app-specific password for the email.	your-email-password

Example .env Configuration

Here's an example .env file for your project: MongoDB URLs
USER_DB_URL=mongodb://localhost:27017/users
BUDGET_DB_URL=mongodb://localhost:27017/budget
TRANSACTION_DB_URL=mongodb://localhost:27017/transaction
TAXESTIMATOR_DB_URL=mongodb://localhost:27017/taxestimator

TAXCALENDER_DB_URL=mongodb://localhost:27017/taxcalculator

REPORT_DB_URL=mongodb://localhost:27017/report

JWT Secret

JWT_SECRET=supersecretkey123

SMTP Configuration

SMTP_HOST=smtp.gmail.com

SMTP_PORT=587

SMTP_USER=your-email@example.com

SMTP_PASS=your-email-password

Steps to Set Up the Environment

- Create a .env file in the root of your backend directory.
- Copy the example configuration above and paste it into the .env file.
- Replace placeholder values (like your-email@example.com and your-email-password) with your actual configuration.

Folder Structure

The project is divided into two main parts: **backend** and **frontend**.

```
TAXPAL_CLONE/
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── index.js
│   ├── package.json
│   └── .env
├── frontendd/
│   ├── src/
│   │   ├── components/
│   │   ├── config/
│   │   ├── assets/
│   └── package.json
└── README.md
```

Key Features

- **Income & Expense Management**: Log and manage all transactions.
- **Categorization & Budgeting**: Assign categories and set budget limits.
- **Tax Estimation Engine**: Region-based automatic tax calculations.
- **Reporting & Export**: Generate and export CSV financial summaries.
- **User Authentication**: Secure login/registration.
- **Reminders & Notifications**: Calendar view for quarterly tax schedules.
- **Responsive Dashboard**: Mobile and desktop-friendly interface.

Project Components

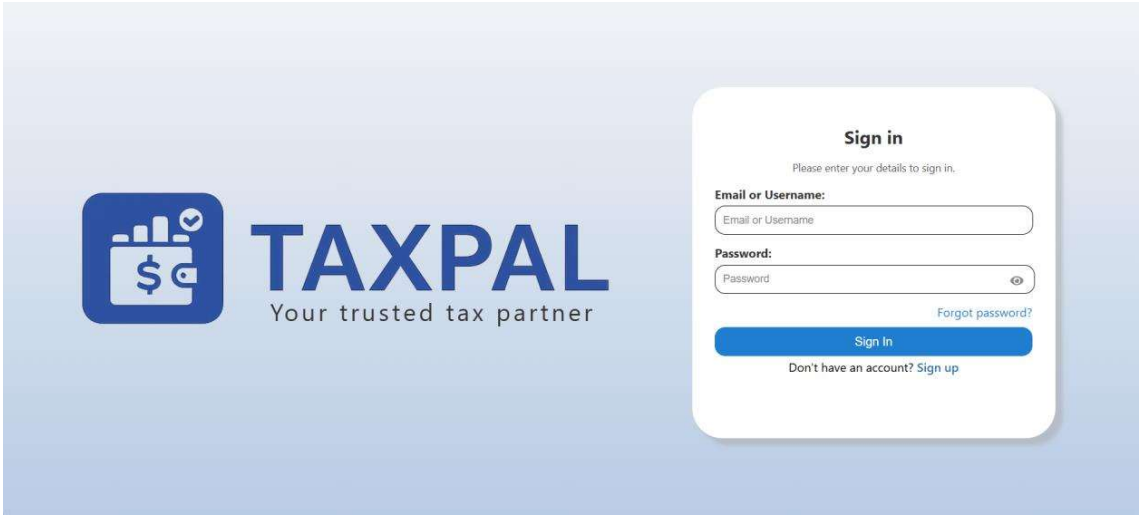
This is a list of key components in the Taxpal project, along with descriptions of their functionality:

Sign in Component

Description: This component handles the user authentication process by validating login credentials (email and password) and allowing access to the personalized dashboard. It is first step that ensures every user's data remains private and protected. The component also supports additional functionality such as redirecting to the **Sign Up** page for new users and providing an option for **password recovery** in case users forget their credentials.

Key features

- **User Authentication:** Verifies user credentials securely against the stored data to prevent unauthorized access.
- **Form Validation:** Ensures users enter valid inputs (for example, email format and non-empty password fields) before submission.
- **Error Handling:** Displays clear error messages when incorrect credentials are provided, making it easy for users to identify and correct mistakes.
- **Password Recovery Link:** Provides a quick navigation option to reset forgotten passwords using email verification.

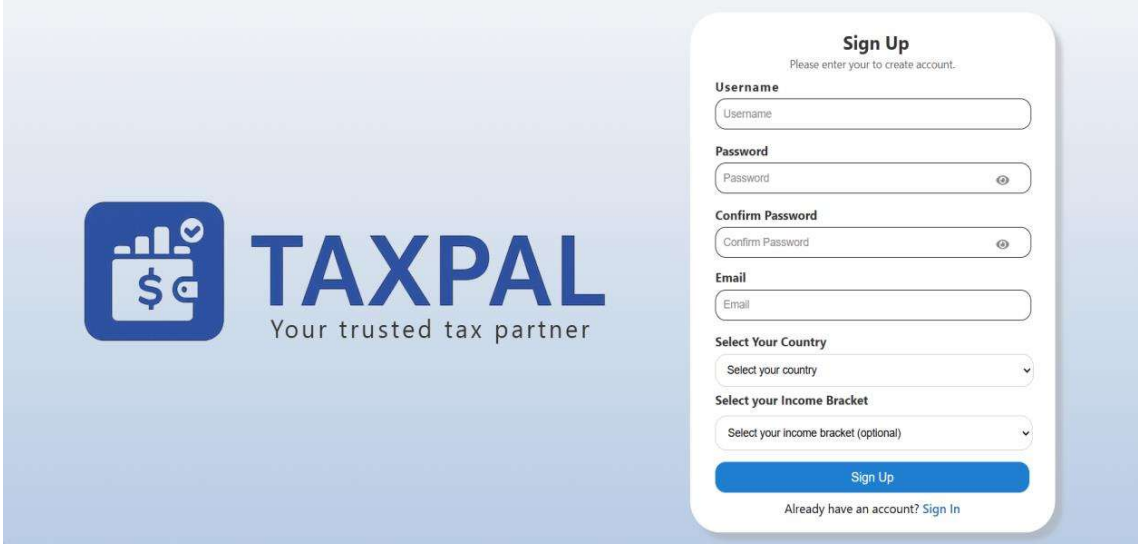


Sign Up Component

Description: This component allows users—especially freelancers and independent workers—to register on the Taxpal platform by providing essential details such as their username, email address, password, and country of residence. Once registered, their information is securely stored in the database, and they can immediately log in to access their personalized dashboard. The Sign Up process ensures that every user has a unique account linked to their financial data. It also includes input validation to prevent issues such as duplicate accounts, weak passwords, or invalid email formats.

Key features

- **User Role Registration:** The form is specifically designed for freelancers and gig workers who want to track their personal income and estimate taxes.
- **Secure Password Handling:** Passwords entered during registration are encrypted before being stored in the database, ensuring data protection and account safety.
- **Duplicate Account Prevention:** If a user tries to register with an email that's already in use, the system displays an appropriate message to avoid confusion.
- **Error Feedback:** Clear messages are shown for issues like missing fields or invalid input, helping users complete registration easily.



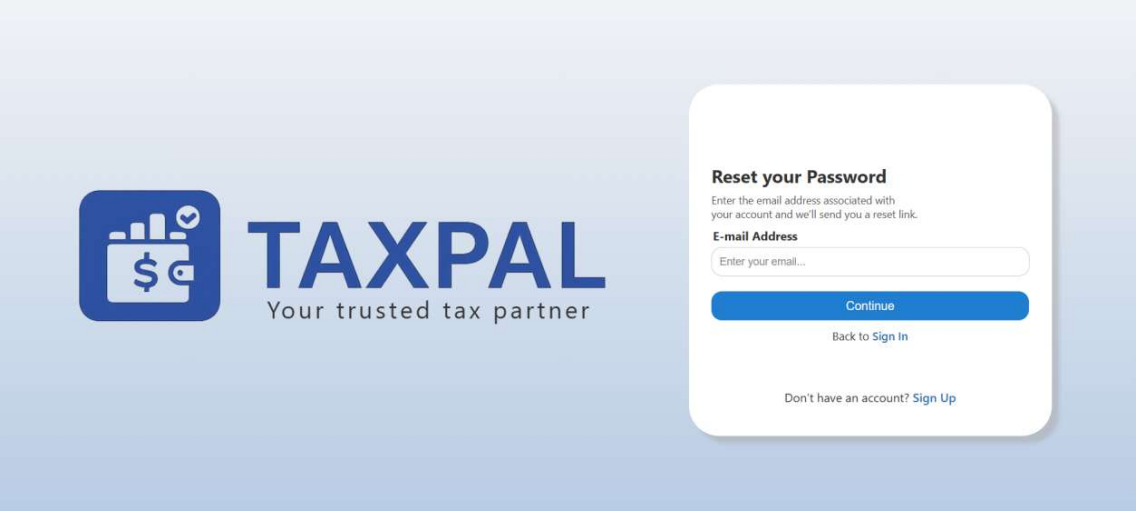
Forgot Password Component

Description: This component allows users to reset their account passwords using an email verification process. When a user clicks on the "Forgot Password" link on the Sign In page, they are prompted to enter their registered email address. Once submitted, the system generates a One-Time Password (OTP) or a secure reset link, which is sent to the user's email address. This ensures that only the account owner can initiate the reset process. After receiving the OTP or link, the user can verify it and set a new password for their Taxpal account.

Key features

- **Email-Based Verification:** Users initiate the password reset by entering their registered email address. The system sends an OTP or reset link to that address.
- **Password Reset Form:** After verifying the OTP or link, users are directed to a form where they can create a new password. Password rules ensure strong and secure credentials.

- **Error Handling:** If a user enters an unregistered email or an invalid OTP, the system displays an appropriate error message, guiding them through the correct steps.
- **Security Measures:** All reset requests are encrypted, and OTPs expire automatically after a limited time to prevent misuse. Passwords are hashed before being saved in the database.

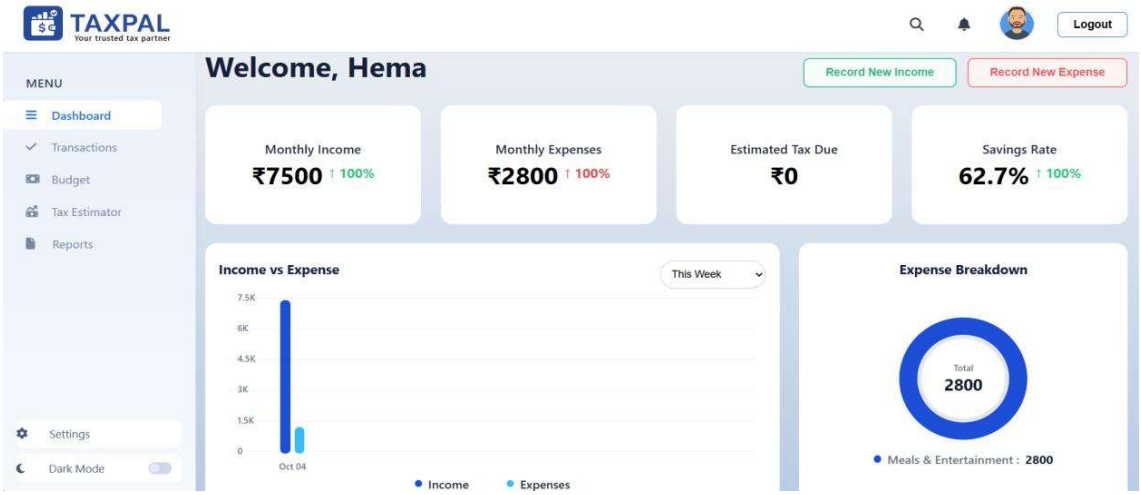


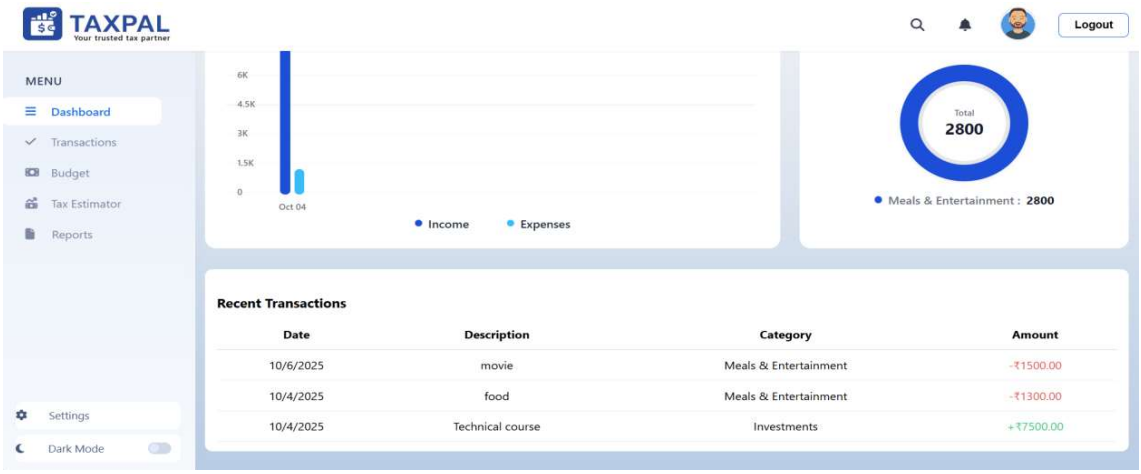
Dashboard Component

Description: The Taxpal Dashboard is designed to give freelancers and gig workers an instant snapshot of their financial health. It acts as a personal finance control panel where users can monitor their transactions, track spending habits, view categorized summaries, and stay updated with their tax estimates. This component integrates data from several modules — including Income & Expense Management, Budgeting, and Tax Estimation — and displays them visually using tables, charts, and summary cards. The layout is simple, responsive, and user-friendly, making it easy to navigate whether on a desktop or mobile device. In addition to viewing summaries, users can perform quick actions directly from the dashboard, such as adding a new income or expense, viewing category breakdowns, or checking reminders for upcoming quarterly tax payments.

Key features

- **Personalized Overview:** Displays the logged-in user's name and key financial details, such as total income, total expenses, and current savings or balance.
- **Budget Tracking:** Provides a visual progress bar or chart showing how much of each budget category (like rent, groceries, or utilities) has been used for the current month.
- **Real-time updates:** When users add or delete a transaction, the dashboard refreshes automatically to display the updated totals and charts.
- **Transaction Summary:** Shows a list or table of recent income and expense entries, including transaction date, category, and amount.
- **Responsive Design:** Buttons or links to add a new transaction, set budgets, or generate reports are easily accessible for smooth navigation.





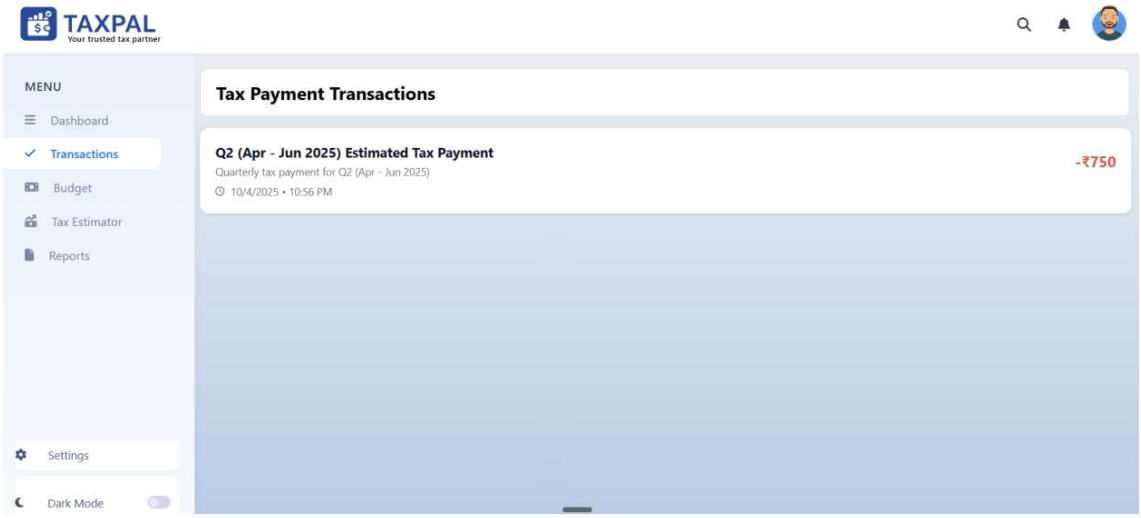
Transactions Component

Description: Once a user signs in and navigates to the Transactions page, they can see a detailed list of all their financial records — both income and expenses. Each transaction displays essential details such as the date, type (income or expense), category, amount, and a short description. This component connects directly to the backend database, retrieving data from the Transactions collection in MongoDB. It ensures that every entry made through the income or expense forms appears instantly in the list. Users can also edit or delete any record if needed, allowing them to keep their financial records accurate and up to date.

Key features

- **Transaction List View:** Displays all transactions made by the user in a neatly formatted table or card layout. Each row or card shows the type, category, date, and amount.
- **Visual Indicators:** Income entries are often shown in green to indicate money coming in. Expense entries are shown in red to indicate money going out. This visual cue helps users instantly understand their financial trends.

- **Pagination or Infinite Scroll:** For users with a large number of records, pagination ensures smooth browsing and prevents data overload.

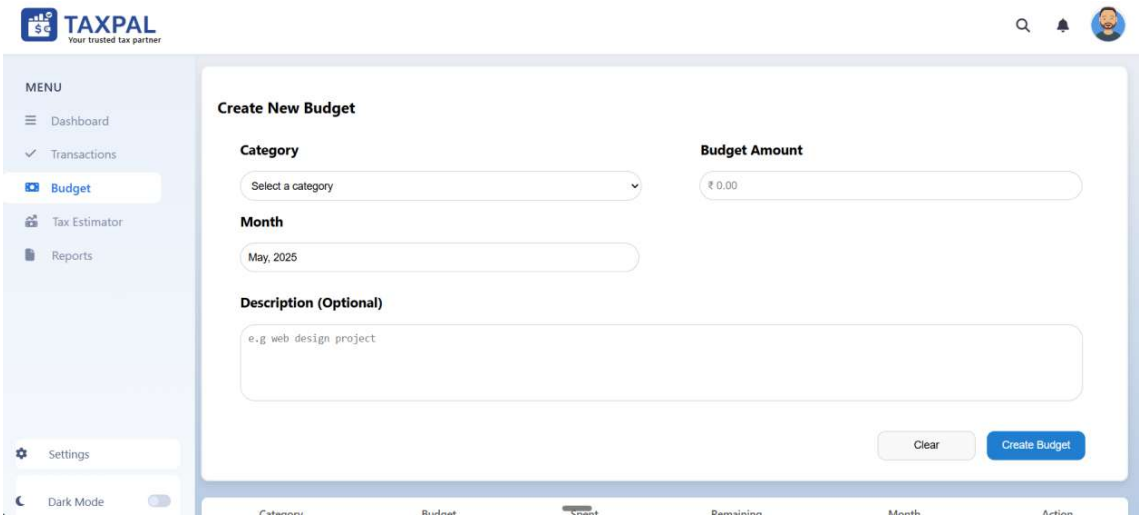


Budget Component

Description: Once users have logged their income and expenses, they can use the Budget component to define how much they want to spend on different categories — such as food, rent, travel, entertainment, or utilities — for a particular month. This component directly connects to the Budgets collection in the database, where each record stores the user ID, category, limit amount, and the month the budget applies to. Whenever a new transaction is added, the system compares it against the budget set for that category and automatically updates the remaining balance. The visual representation of each budget — usually shown through bars, charts, or color indicators — allows users to quickly identify whether they are within their spending limits or approaching them.

Key features

- **Set Monthly Budgets:** Users can define a maximum spending limit for each expense category for the current month (for example, ₹5000 for Food, ₹2000 for Transportation).
- **Category-wise Tracking:** The component monitors each category's total spending and compares it against the set limit, updating progress bars or percentage indicators in real time.
- **Edit and Update Budgets:** Users can modify their budgets any time during the month — for instance, if they decide to reduce entertainment expenses or increase travel allowances.
- **Monthly Overview Chart:** A summary chart or graph displays total monthly spending versus the total monthly budget, helping users understand their overall financial discipline.
- **Automatic Calculations:** Every time a new expense is logged through the Transactions component, the system automatically recalculates the remaining budget for that category and updates the visuals instantly.

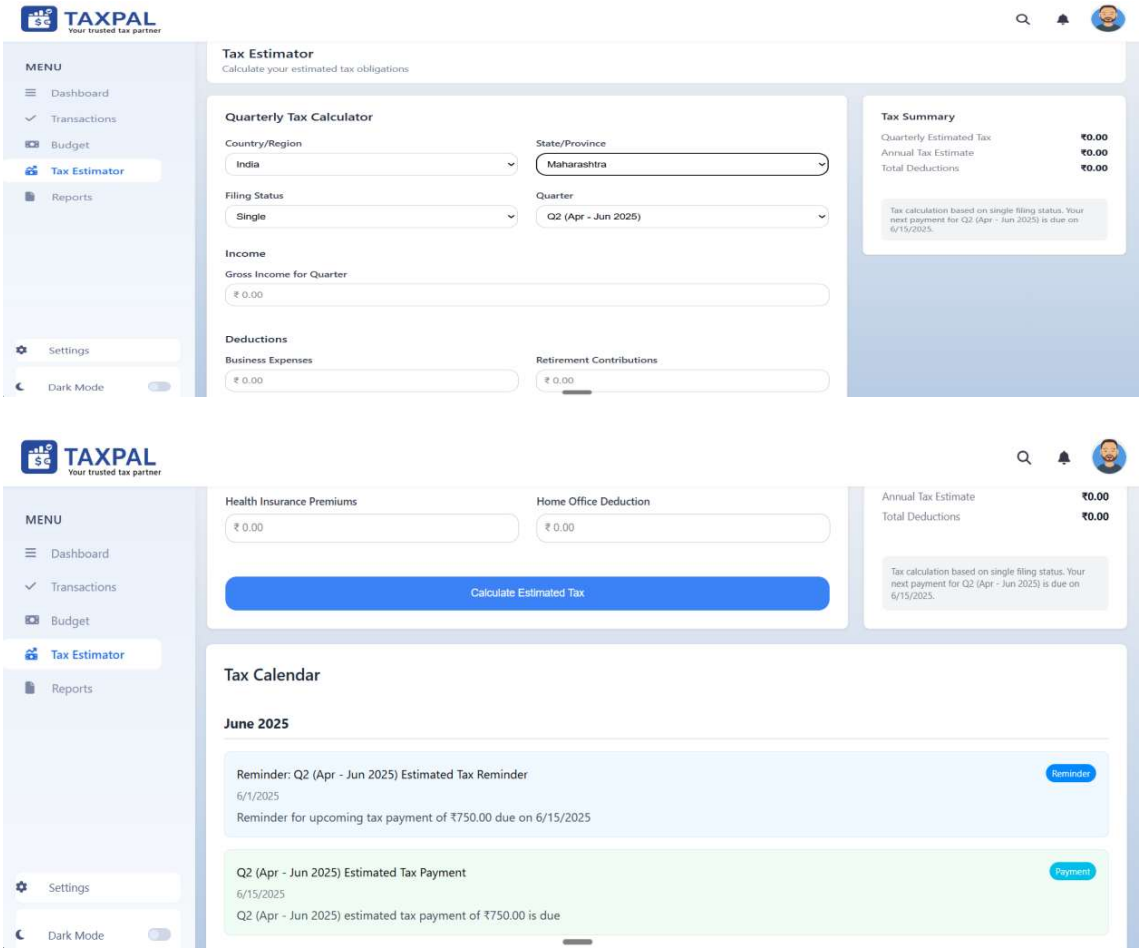


Tax Estimator Component

Description: Freelancers often have multiple sources of income, and keeping track of how much tax they owe can be confusing. The Tax Estimator component simplifies this process by analyzing all logged income transactions, applying relevant regional tax rates, and instantly generating an estimated tax amount for the selected period. When users open the Tax Estimator page from the sidebar, they can choose their country or region, select a time frame (monthly, quarterly, or yearly), and click a button to calculate their tax. Based on the income data stored in the database, Taxpal's calculation logic automatically applies the correct tax brackets to determine the total estimated amount. The system then displays the calculated tax along with a clear breakdown, showing how the amount was derived (for example, "10% for the first ₹2,50,000, 20% for the next ₹2,50,000," etc.). Users can also view a calendar of quarterly due dates, helping them remember when their next tax payment is approaching.

Key features

- **Automatic Tax Calculation:** The component automatically calculates estimated taxes based on logged income data and applicable tax slabs for the user's region.
- **Region Selection:** Users can select their country or region from a dropdown menu. The tax rules and slabs adjust accordingly, ensuring accurate results.
- **Quarterly Tax Estimation:** Generates tax estimates for each fiscal quarter (Q1, Q2, Q3, and Q4), helping users plan their payments in advance.
- **Calendar View for Due Dates:** Includes a calendar or reminder section displaying upcoming quarterly tax deadlines, so users never miss a payment.
- **Error Handling and Validation:** If users haven't logged any income or selected a region, the system prompts them with clear messages such as *"Please enter your income details first"* or *"Select a region to estimate tax."*



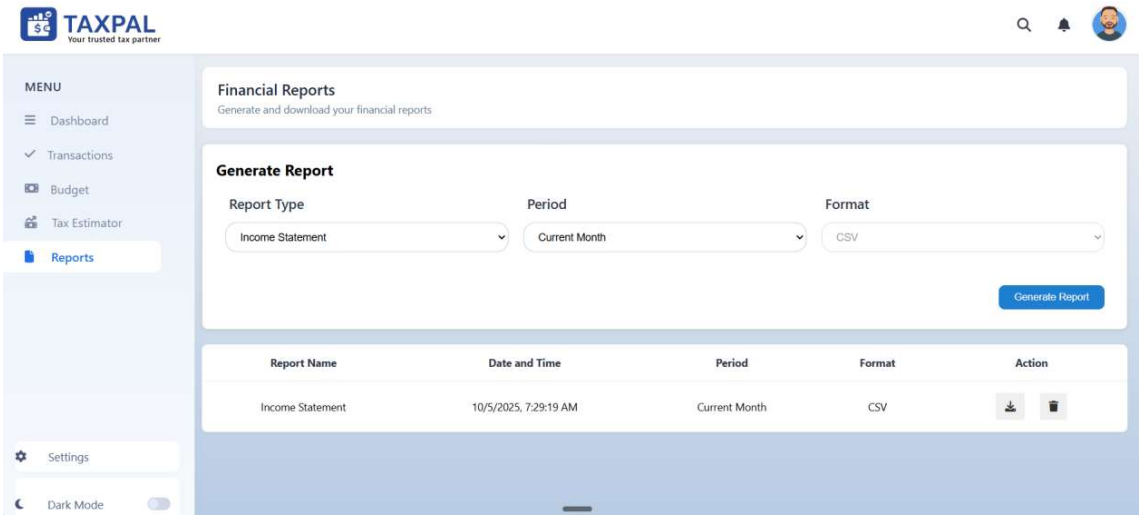
Reports Component

Description: Managing finances effectively requires not just tracking transactions, but also understanding patterns and outcomes over time. The Reports component in Taxpal is designed for exactly that purpose. It gathers data from multiple parts of the application — including transactions, budgets, and tax estimates — and organizes them into meaningful summaries. Users can generate reports for different time frames such as monthly, quarterly, or yearly, depending on their needs. For instance, a freelancer may want to generate a quarterly report that combines income, expenses, and the corresponding tax estimate. Moreover, users can download these reports in CSV format, enabling them to maintain proper records or share them with accountants and financial advisors.

Key features

- **Custom Report Generation:** Users can choose the reporting period — such as a specific month, quarter, or year — and generate a detailed summary of their financial activities.
- **Download & Storage:** Each report is saved with a unique file name and timestamp in the database (Reports collection), allowing users to download it anytime later.

- **Comprehensive Data Summary:** The report includes total income, total expenses, net savings, budget comparisons, and tax estimates for the selected period.
- **Tax Summary Integration:** The report includes an automatically calculated tax estimate section for the same period, ensuring users have everything they need in one place.



Settings Component

Description: The Settings component in Taxpal acts as a central hub where users can manage their personal information, category preferences, notification choices, and account security. It allows every user to tailor the application according to their individual needs, ensuring both convenience and control. When users open the Settings page from the sidebar, they see four well-structured sections — Profile, Categories, Notification, and Security — each serving a specific purpose.

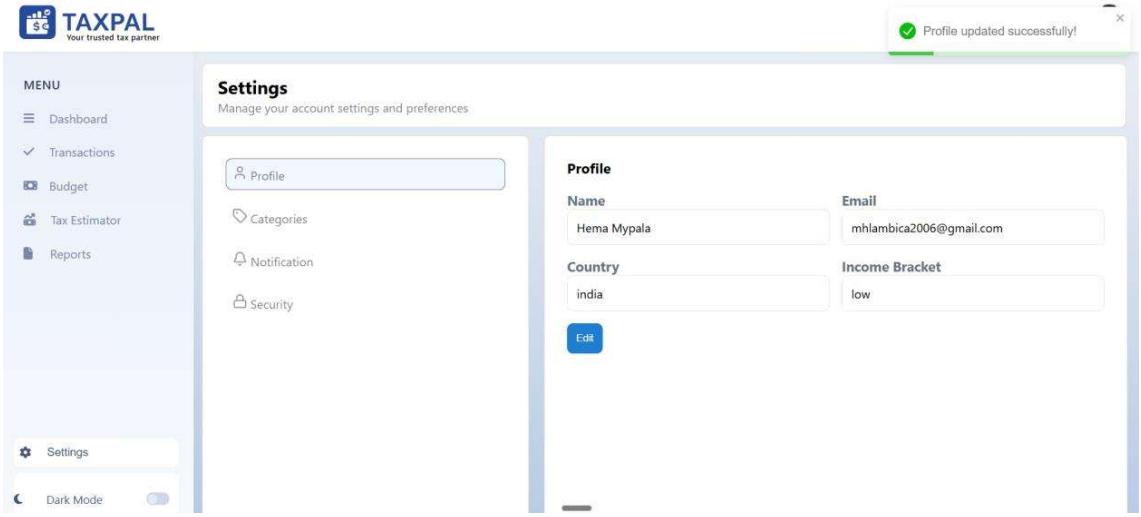
1. **<u>Profile Section</u>**

   Description: The Profile section allows users to view and update their personal

   details such as Name, Email, Country and Income Bracket. This ensures that their account information stays accurate and up to date. Whenever users click the Edit button, they can modify their details and save the changes instantly. These updates are securely stored in the database and reflected across the application — including in tax estimations and reports.

   Key features

   - View and edit personal information (Name, Email, Country, Income Bracket).
   - Data is validated and securely updated in the backend.
   - Real-time reflection of updated information across all connected modules.
   - Clean and minimal form layout for easy editing.
   - Confirmation messages after updates, e.g., "Profile updated successfully."

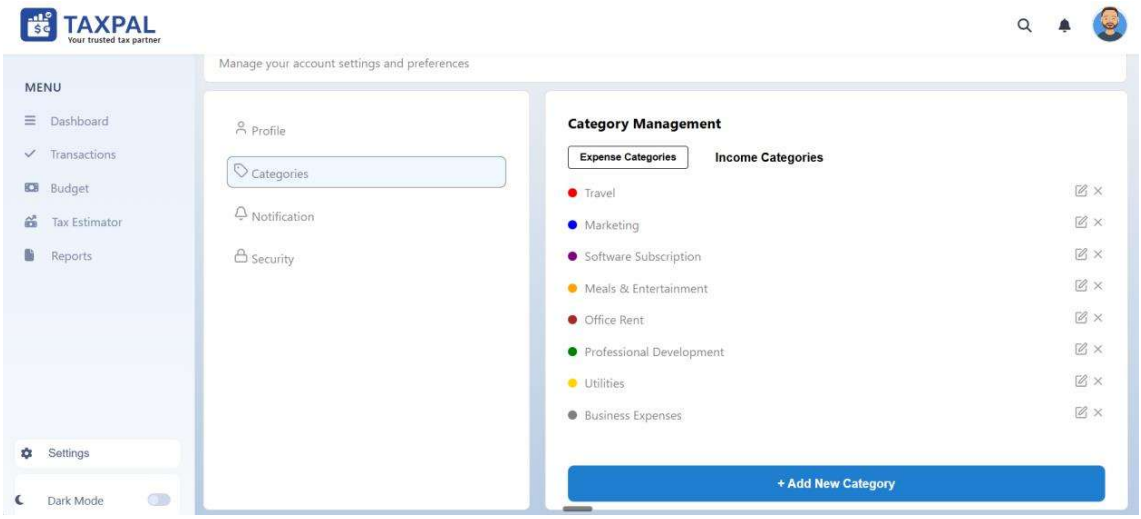


2. Categories Section

Description: The Categories section helps users customize their income and expense categories. This feature ensures smoother and more personalized transaction management. Users can add new categories under Income or Expenses, which automatically appear in dropdown lists while adding new transactions. This reduces manual typing and helps maintain organized financial records.

Key features

- Add, edit, or remove custom categories for both income and expenses.
- Automatically integrates new categories into the transaction forms.
- Helps maintain organized and consistent data entry.
- Displays all existing categories in an easy-to-view list format.
- Instant feedback on successful category creation or deletion.

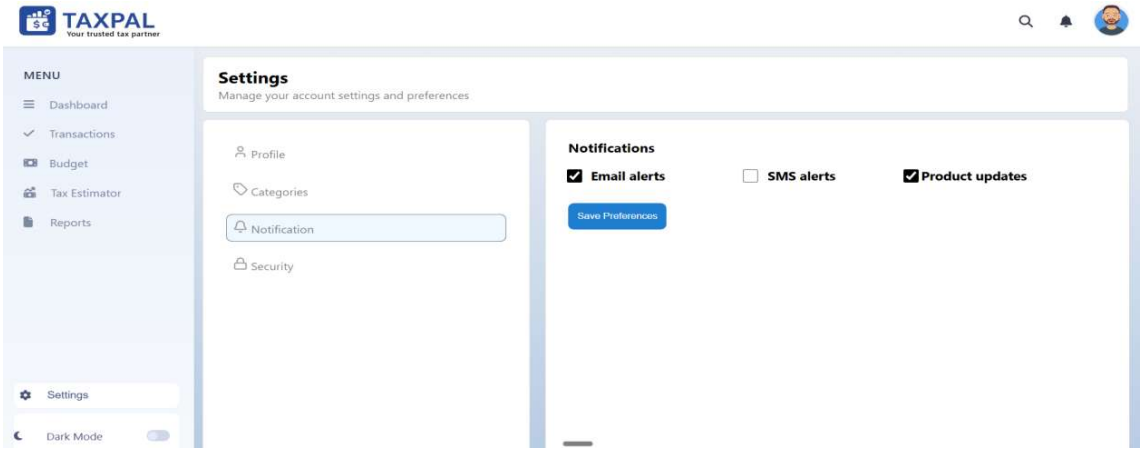


3. Notification Section

Description: The Notification section allows users to choose how they want to receive alerts and updates from Taxpal. Since users may prefer different modes of communication, this feature gives them control over how they stay informed about important activities or updates. Users can select options such as Email Alerts, SMS Alerts, and Product Updates, and save their preferences with a single click.

Key features

- Multiple notification types: Email Alerts, SMS Alerts, Product Updates.
- Preferences saved securely and applied immediately.
- Clear confirmation messages after saving preferences, e.g., "Notification settings saved successfully."
- Simple toggle-style or checkbox interface for easy selection.
- Enhances user engagement while respecting their preferred communication channels.

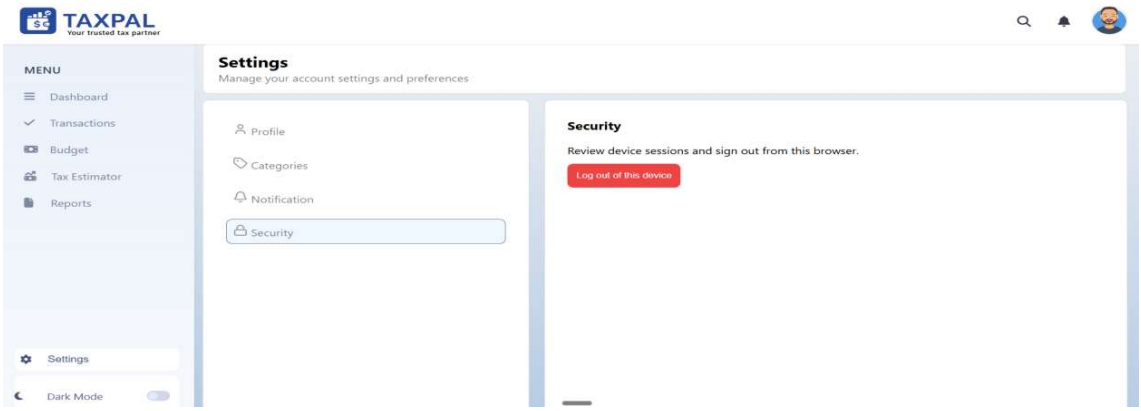


4. Security Section

Description: The Security section focuses on safeguarding user accounts. It currently includes an option that allows users to log out securely from the device they are using. This ensures that their account remains protected, especially when accessed from public or shared systems. In future updates, this section can be expanded to include additional features such as password changes or multi-device logout functionality.

Key features

- **Logout from Current Device:** Ends the active session securely and redirects to the login page.
- Ensures privacy and prevents unauthorized access.
- Visible confirmation message after logout, e.g., "You have been logged out successfully."
- Prepares the foundation for future security enhancements.



API Components

Base URL

Text http://localhost:5000/api

Authentication APIs:

POST /auth/register - User registration

POST /auth/login - User login

POST /auth/verify-otp - Email verification

POST /auth/resend-otp - Resend OTP

POST /auth/send-reset-otp - Send password reset OTP

POST /auth/reset-password - Reset password

Transaction APIs:

GET /transactions - Get user transactions

POST /transactions - Create new transaction (income/expense)

Tax Estimation APIs:

Tax estimation endpoint (in TaxEstimator.jsx)

Headers used:

Authorization: Bearer <token> for authenticated requests

Register User

Route: POST /auth/register

{

"name": "string",

"email": "string",

"password": "string",

"country": "string",

"income": "string"

}

Response

"success": true,

"message": "Registration successful! Please verify your email.",

"emailPreview": "string"

}

Login

http

POST /auth/login

Content-Type: application/json

{

"email": "string",

"password": "string"

}

Response:

Json

```
{
  “success”: true,
  “token”: “jwt_token”,
  “user”: {
    “id”: “string”,
    “name”: “string”,
    “email”: “string”,
    “country”: “string”,
    “income”: “string”
  }
}
```

Verify Email OTP

http

Apply to index.css

```
POST /auth/verify-otp
Content-Type: application/json
{
  “email”: “string”,
  “otp”: “string”
}
```

Resend OTP

http

Apply to index.css

```
POST /auth/resend-otp
Content-Type: application/json
{
  “email”: “string”
```

}

Send Reset Password OTP

http

Apply to index.css

POST /auth/send-reset-otp

Content-Type: application/json

{

“email”: “string”

}

Reset Password

http

Apply to index.css

POST /auth/reset-password

Content-Type: application/json

{

“email”: “string”,

“otp”: “string”,

“newPassword”: “string”

}

Transaction Endpoints

Get Transactions

http

Apply to index.css

GET /transactions

Authorization: Bearer <token>

Response:

Json

Apply to index.css

```
{
  “success”: true,
  “items”: [
    {
      “_id”: “string”,
      “type”: “income|expense”,
      “description”: “string”,
      “category”: “string”,
      “amount”: “number”,
      “date”: “string”,
      “notes”: “string”,
      “user”: “string”
    }
  ]
}
```

Create Transaction

http

Apply to index.css

```
POST /transactions
Authorization: Bearer <token>
Content-Type: application/json
{
  “type”: “income|expense”,
  “description”: “string”,
  “category”: “string”,
  “amount”: “number”,
  “date”: “string”,
  “notes”: “string”
```

}

Tax Estimation Endpoints

Create Tax Estimate

http

Apply to index.css

POST /tax-estimate

Authorization: Bearer <token>

Content-Type: application/json

{

“income”: “number”,

“expenses”: “number”,

“country”: “string”

}

Database Schema

Users Collection

Javascript

Apply to index.css

{

_id: ObjectId,

Name: String,

Email: String (unique),

passwordHash: String,

country: String,

income: String,

isEmailVerified: Boolean,

createdAt: Date,

updatedAt: Date

}

Transactions Collection

Javascript

Apply to index.css

```
{
  _id: ObjectId,
  User: ObjectId (ref: User),
  Type: String (enum: [‘income’, ‘expense’]),
  Description: String,
  Category: String,
  Amount: Number,
  Date: Date,
  Notes: String,
  createdAt: Date,
  updatedAt: Date
}
```

TaxEstimates Collection

Javascript

Apply to index.css

```
{
  _id: ObjectId,
  User: ObjectId (ref: User),
  Income: Number,
  Expenses: Number,
  estimatedTax: Number,
  country: String,
  createdAt: Date,
  updatedAt: Date
```

}

Error Handling and Troubleshooting

Effective error handling and troubleshooting ensure the smooth operation of the Right Resource Fit platform. Below are some common errors that users and developers may encounter, along with steps for troubleshooting and resolving them.

Gmail SMTP Not Working

Error Description: The platform uses SMTP to send email notifications (e.g., OTPs, confirmation emails). If SMTP fails, emails won't be sent.

Troubleshooting Steps:

Check Gmail Settings: Ensure that less secure app access is enabled in your Gmail account. Gmail may block connections from apps it considers less secure.

- Navigate to your Google Account settings.
- Under "Less secure app access," turn on access.

Check Credentials: Double-check the email and password provided in the configuration to ensure they are correct.

- Check SMTP Settings:
- SMTP Server: smtp.gmail.com
- Port: 587 (for TLS)
- Ensure the secure: true flag is set in your SMTP configuration.
- Check 2-Step Verification: If you have 2-step verification enabled, you need to generate and use an App Password instead of your regular Gmail password.

Port Conflicts

Error Description: The application may fail to start due to port conflicts (e.g., the server tries to bind to port 3000, but it's already being used by another application).

Troubleshooting Steps:

Check Which Service is Using the Port:

On Linux/macOS: you can use the following command to check:

Lsof -I :3000

On Windows, use:

Netstat -ano | findstr :3000

This will show you which process is using the port.

Stop the Conflicting Process:

If you find a process using the port, you can stop it by identifying its PID and killing it.

On Linux/macOS, run:

Kill -9 <PID>

On Windows, use:

Taskkill /PID <PID> /F

Change the Port: If you can't stop the conflicting process or prefer to use a different port, update your

Application's configuration file to use a new port number (e.g., 3001 or 8080). Example (Node.js Express App):

Database Connection Failures

Error Description: The application may fail to connect to the database due to incorrect configuration or service Unavailability.

Troubleshooting Steps:

- Check Database Credentials: Ensure that the database username, password, and connection string in the Configuration are correct.
- For MongoDB, check the connection string format and the database's host URL.
- Ensure Database is Running: Make sure that the MongoDB or other database service is running on your local Machine or remote server.
- Check Database Access Permissions: Ensure that the database user has sufficient permissions to perform Operations (e.g., read, write).

API Request Failures

Error Description: API requests may fail due to misconfiguration or incorrect endpoint usage.

Troubleshooting Steps:

- Check Endpoint URL: Ensure that the correct API endpoint is being used. Check the route and URL in the backend and the client-side code.
- Check API Authentication: If your API requires authentication (e.g., API keys or tokens), ensure that these Credentials are correctly included in the request headers.
- Examine Response Codes: Check the HTTP status codes returned by the API. Common errors include:
- 404 (Not Found): The requested endpoint is incorrect or missing.

- 500 (Internal Server Error): There's an issue with the server; check logs for more details.
- 401 (Unauthorized): Authentication issues, such as missing or invalid API keys.

Authentication Issues (JWT or Session Timeout)

Error Description: Users may experience issues logging in or maintaining sessions due to token expiration or invalid credentials.

Troubleshooting Steps:

Check Token Expiry: If you are using JWT tokens, ensure that the token hasn't expired. JWT tokens typically have an expiration time encoded in them.

Ensure Correct Login Credentials: Check that the user is providing the correct username and password and that your backend is validating these correctly.

Check Session Storage: If using sessions, ensure the session cookie is correctly stored and sent with each subsequent request.

Frontend Rendering Issues

Error Description: The frontend UI may fail to load or display incorrectly due to errors in rendering or incorrect data passed to components.

Troubleshooting Steps:

- Check JavaScript Errors: Use the browser's developer tools (F12) to check for JavaScript errors in the console.
- Look for error messages related to missing variables, incorrect function calls, or syntax issues.
- Inspect DOM Elements: Ensure the correct data is being passed to your UI components. Use console.log() to log the data being passed into templates or components.
- Ensure Dependencies Are Installed: Check if all the required frontend dependencies are installed (e.g.,React.js) Run npm install or yarn install to install any missing packages.

General Troubleshooting Tips

Clear Cache: Sometimes, browser or application caches can cause issues. Clear the cache or try running the application in incognito mode.

Check Server Logs: Always check the server logs for detailed error messages. They can provide more context on what's going wrong.

Use Debugging Tools: Utilize debugging tools like Chrome DevTools, Visual Studio Code debugger, or backend-specific debuggers to step through the code and identify issues

Maintenance and Support: Procedures for Regular Maintenance Backing Up the Database.

- Use MongoDB's built-in backup tools or cloud-based solutions like MongoDB Atlas to schedule regular backups.
- Store backups securely in a cloud storage service (e.g., AWS S3, Google Cloud Storage) to prevent data loss.
- Rotating API Keys or Credentials

Regularly update sensitive credentials stored in environment variables (e.g., .env files).

- Use tools like AWS Secrets Manager or Azure Key Vault for secure storage.
- Monitoring Logs and Server Health
- Implement logging with tools like Winston or ELK Stack for application logs.
- Use monitoring tools like New Relic, Prometheus, or Datadog to track server performance and uptime.

How Users Can Report Issues or Request Support

- Provide a dedicated Support Email Address (e.g., support@rightresourcefit.com).
- Include a link to a Support Pagewhere users can:
- Submit issue reports.
- Access self-help guides or FAQs.
- Set up a ticketing system using tools like Zendesk or Jira for efficient issue tracking.

Version Control and Updates:

Using Git for Version Control

Branching Strategies:

- Use the main branch for production-ready code.
- Create feature branches for new features or bug fixes.
- Use release branches for testing updates before merging into the main branch.

Commit Messages:

- Follow a standard format such as [feature]: Add user authentication or [fix]: Resolve login bug.
- Applying Updates and Migrations Safely
- Test updates in a staging environment before deploying to production.

- Perform database migrations using tools like Mongoose migration scripts